

**Retrieval-Augmented Generation for Educational AI Chatbot in Geotechnical Engineering:
Bridging Theory and Construction Practice with Domain-Specific Knowledge**

Andrew Fagan

Research Assistant

Department of Computer and Information Science

State University of New York Polytechnic Institute

100 Seymour Rd, Utica, NY 13502

Email: fagana@sunypoly.edu

Asif Ahmed, PhD, PE

Assistant Professor

College of Engineering

State University of New York Polytechnic Institute

100 Seymour Rd, Utica, NY 13502

Email: ahmeda3@sunypoly.edu

Word Count: 6552 words + 1 table (250 words per Table) = 6802 words

ABSTRACT

The evolving landscape of civil engineering education necessitates innovative tools to bridge the traditional gap between theoretical design principles and real-world construction challenges. This study presents the development and preliminary validation of "Asif Ahmed's AI-Answerer," a web-based conversational agent designed to assist students in geotechnical engineering courses by providing answers exclusively from instructor-provided materials. The system is built using the Streamlit framework for an intuitive user interface and leverages Google's Gemini Large Language Model (LLM) for natural language understanding and generation. Its core knowledge base comprises Professor Ahmed's textbooks, YouTube video transcripts, and supplementary PDF documents. A simplified Retrieval-Augmented Generation (RAG) approach is implemented, where the LLM is meticulously prompted to ensure responses are strictly grounded in the selected document, effectively mitigating hallucination and maintaining instructional fidelity. The chatbot's reliability was validated through a series of domain-specific queries, demonstrating its ability to accurately and relevantly answer questions on topics such as deep foundations, dewatering, and shallow foundations. This research highlights the potential of domain-specific AI chatbots in enhancing student comprehension, addressing practical construction challenges within academic curricula, and preparing future civil engineers for industry demands.

Keywords: Educational Chatbot, Large Language Model (LLM), Retrieval-Augmented Generation (RAG), Geotechnical Engineering Education, Streamlit, Google Gemini, Domain-Specific AI, Construction Practice.

INTRODUCTION

Civil engineering design courses traditionally focus on structural, geotechnical, and transportation-related design principles, ensuring that students develop a strong technical foundation. However, the real-world implementation of these designs in the construction phase is often more complex than what is covered in textbooks or classroom lectures. Construction challenges, including sequencing, material procurement, site constraints, workforce coordination, environmental impacts, and unexpected on-site conditions, significantly influence the successful execution of a project. Yet, these aspects remain underemphasized in the conventional civil engineering curriculum [1]. As a result, students graduate with a deep understanding of theoretical design principles but limited exposure to how these designs are practically implemented in the field. This knowledge gap often becomes apparent when graduates enter the workforce and struggle to bridge the divide between design intent and construction reality.

One of the key limitations of existing teaching methods is that they fail to simulate the dynamic and often unpredictable nature of construction projects. Unlike the controlled environment of design calculations, construction sites present challenges such as weather-related delays, supply chain disruptions, soil variability, and labor shortages [2]. These unpredictable elements require engineers to think beyond textbook solutions and adapt quickly to changing circumstances. Additionally, construction sequencing plays a crucial role in ensuring efficient project execution, yet students rarely get the opportunity to engage with realistic scenarios where they must plan and adjust construction processes. Without this exposure, students may find themselves unprepared to handle the fast-paced decision-making and interdisciplinary coordination required in the construction industry. This gap between design education and real-world construction challenges not only affect students' preparedness but also contributes to inefficiencies in the industry, where young engineers take time to develop the necessary practical skills [3]. To address this issue, there is a need for innovative educational tools that integrate construction considerations into the design curriculum, enabling students to better understand how their designs translate into built structures.

To bridge this gap, we propose the development of an AI-driven tool that will assist students in understanding construction sequences and challenges within the context of design courses. This tool could take the form of an interactive AI chat assistant or a more immersive AI-powered simulation, providing students with real-time guidance on construction-related aspects of their design projects. By integrating AI assistance, students will be able to explore various construction challenges, receive step-by-step construction sequencing guidance, and engage with realistic case studies that highlight common issues encountered on construction sites. The AI tool could allow students to input their design specifications and receive feedback on potential construction difficulties, such as required temporary structures, excavation challenges, or sequencing conflicts. This interactive engagement will encourage students to think beyond design principles and develop a construction-aware mindset early in their academic journey.

One potential feature of the AI tool will be the ability to generate customized construction scenarios based on students' design inputs. For example, if a student is designing a deep foundation system, the AI could provide insights into site preparation, equipment selection, dewatering needs, and soil stabilization techniques. If a student is designing a retaining wall, the AI could highlight potential construction challenges such as formwork setup, backfilling sequences, and drainage considerations. Additionally, the AI tool could include a decision-making module where students are presented with real-world construction dilemmas and must choose the best course of action, simulating the problem-solving skills required in the industry. This approach will not only enhance students' understanding of construction complexities but also

prepare them to be more effective in professional settings where they must collaborate with contractors, project managers, and other stakeholders. By infusing AI assistance into civil engineering design courses, we aim to create a more holistic educational experience that equips students with the knowledge and skills necessary to navigate the real-world challenges of construction. Moreover, the program does not have the luxury to increase the required credit hours for graduation; as such, infusing construction-related concepts into existing design courses is the most practical and efficient solution. In this project, we've attempted to infuse the construction related concepts for two geotechnical engineering courses (i) Foundation Analysis and Design, and (ii) Design of Earth Structures. As such, the goal of the study is to develop AI based tool for construction scenarios of the geotechnical engineers.

BACKGROUND

Why is Now the Ideal Time to Prioritize Construction in Geotechnical Engineering

We are at a pivotal moment in history to enhance civil engineering education by emphasizing stronger competencies in construction. This urgency stems from the passing of the U.S. Infrastructure Investment and Jobs Act — the largest infrastructure investment in U.S. history, approved by Congress in 2021 [4]. This \$1.2 trillion USD initiative is driving rapid growth in the construction industry, creating significant demand for skilled civil engineering graduates. In 2022, the construction sector employed 4.7% of the U.S. workforce, making it one of the most substantial contributors to the national economy [5]. Moreover, employment in the construction industry is projected to increase by 4% from 2022 to 2032, reinforcing the need for a well-prepared workforce to meet industry demands [5]. In New York state, Governor Kathy Hochul announced an investment of \$350 million for workforce development, including the creation of the Office of Strategic Workforce Development (OSWD) within Empire State Development (ESD) in April 2022 [6]. Through OSWD, ESD has selected seven (7) statewide priority industries where 'Construction' is listed with other 6 disciplines.

AI Powered Chatbots

Artificial intelligence (AI)-powered chatbots have witnessed substantial growth in academic settings, evolving from basic question-answer systems to sophisticated educational assistants. These AI-driven tools have been deployed across a wide range of functions in higher education, including delivering instant feedback on assignments, responding to course-specific inquiries, streamlining administrative tasks such as enrollment, and distributing timely campus-related information [7-10]. A noteworthy evolution within this space is the rise of mentoring bots—specialized systems that provide continuous, round-the-clock instructional support. These bots have proven effective in alleviating faculty and teaching assistant (TA) workloads while improving accessibility for learners [7], [11].

Several domain-specific educational chatbots illustrate the practical benefits of such tools. For example, *Code Tutor* (Hobert, 2019) was developed to assist novice programmers by guiding them through coding exercises, offering step-by-step feedback, and evaluating open-ended responses through natural language interactions [12]. *Python-Bot* (Okonkwo and Ade-Ibijola, 2020) similarly supports the teaching of Python syntax and programming logic to beginners [13]. In the realm of language acquisition, *Mobile Chatbot* (Pham et al., 2018) incorporates gamification techniques and diverse linguistic tasks to boost learner motivation and retention [14]. These examples underscore the pedagogical versatility of chatbot systems in diverse educational contexts.

Emergence of Large Language Model (LLM)

While these conventional chatbot applications have shown promise, the emergence of Large Language Models (LLMs) marks a fundamental shift in AI's role in education. Models like GPT-4, Claude 3.5, and Llama-3 leverage massive datasets and advanced architectures to offer nuanced, human-like language understanding and generation. Their integration into education heralds new opportunities for personalized instruction, enhanced engagement, and improved support for complex problem-solving.

LLMs have been rigorously evaluated across STEM disciplines, including engineering, medicine, and computer science, through standardized testing scenarios. These assessments reveal important patterns regarding LLM capabilities and limitations. For instance, Pursnani et al. [15] analyzed ChatGPT's performance on the U.S. Fundamentals of Engineering (FE) Exam, demonstrating that strategic prompt engineering improves model accuracy, though domain-specific reasoning gaps persist. VarastehNezhad et al. [16] benchmarked GPT-4, Claude 3.5, and Llama-3 on graduate entrance exams in computer science, highlighting the models' aptitude for structured tasks but reduced performance in higher-order logical reasoning. Maitland et al. [17] assessed GPT-4 on medical board exams, noting high scores in multiple-choice formats but common errors in open-ended responses. Honig et al. [18] further investigated student interaction with GPT-based AI tutors in engineering courses, advocating for adaptive, hybrid learning environments that combine AI feedback with human oversight. These findings underscore the importance of domain-specific customization to fully harness the educational value of LLMs. In specialized fields like geotechnical engineering—where problems often require the application of fundamental principles, empirical correlations, and contextual judgment—LLMs must be carefully fine-tuned to handle technical vocabulary, structured calculations, and engineering design logic.

LLM involvement in Geotechnical Engineering

Emerging applications already demonstrate the feasibility of this approach. Xu et al. [19] developed *GeoLLM*, a domain-specific model tailored for automating geotechnical design workflows. GeoLLM has shown promising capabilities in solving foundational geotechnical tasks, such as bearing capacity and settlement computations, effectively processing technical inputs and delivering interpretable outputs. Similarly, Wu et al. [20] explored the integration of LLMs into geotechnical project workflows, noting that these models can aid in data interpretation, support preliminary design decisions, and reduce cognitive load for practitioners. In light of these advancements, the development of specialized LLMs for geotechnical engineering holds significant promise for reshaping engineering education and practice. By addressing both the cognitive demands and practical workflows unique to the discipline, such models can serve as intelligent tutors, design assistants, and decision-support tools—ultimately bridging gaps between theoretical learning, real-world application, and professional preparedness.

MATERIALS

- Computer
- VSCode (python extention)
- Github account
- Internet access
- Engineering documents (textbook PDFs, PDFs, YouTube links)
- Google account

- Streamlit account

METHODOLOGY

The system architecture of the developed chatbot is structured around a client-server model, utilizing a Python-based ecosystem. The core components include a web-based user interface, a backend processing module for data ingestion and LLM interaction, and a carefully curated knowledge base.

Frontend (Streamlit):

The user interface is developed using Streamlit, an open-source Python library renowned for its ability to rapidly build and share data applications with minimal code [21]. Streamlit facilitates an intuitive chat interface, allowing students to select specific course documents and interact with the AI assistant. It manages user input, displays chat history, and provides controls for session management. Streamlit has been successfully employed for various educational applications due to its simplicity and efficiency[8][9][10].

Backend and AI Integration (Python, Google Gemini)

The backend logic, entirely written in Python, handles file reading, context preparation, and interaction with the Google Gemini API. Google Gemini, a multimodal LLM from Google DeepMind, is chosen for its advanced reasoning and generation capabilities, making it suitable for domain-specific question-answering tasks[22]. The integration ensures secure access to the Gemini model via API keys managed through environment variables (dotenv).

Knowledge Base

The foundational strength of the chatbot lies in its exclusive reliance on Professor Asif Ahmed's course materials. This comprehensive knowledge base includes:

- Textbooks (converted to PDF).
- YouTube video transcripts (processed into PDF format).
- Supplementary PDF documents.

All source materials are maintained in a structured directory (AsifAhmed/) for easy access and processing for future documents.

IMPLEMENTATION

The implementation follows a Retrieval-Augmented Generation (RAG) approach, albeit a simplified one, where the "retrieval" step involves providing the entire content of a user-selected document as context to the LLM.

Data Ingestion and Preprocessing

The project handles diverse educational content, specifically PDF documents that may originate from textbooks or converted YouTube transcripts. The pypdf library is utilized for robust PDF parsing and text extraction. At first, the read_file(option) function dynamically selects the relevant PDF based on user choice from a Streamlit dropdown menu (Figure 1)

```

def read_file(option):
    if option == "Chapter 11 - Deep Foundations":
        file = "Deep Foundation Cudotu.pdf"
    elif option == "Shallow Foundation - Step by Step":
        file = "shallow foundation step by step.pdf"
    elif option == "Types of Foundations Footings in Building Construction Video":
        file = "Types of Foundations Footings in Building Construction.pdf"
    elif option == "Shallow Foundations and Their Types Foundations in Building Video":
        file = "Shallow Foundations and Their Types Foundations in Building 2.pdf"
    elif option == "Foundation Insulation Effectiveness Frost Protected Shallow Foundations Video":
        file = "Foundation Insulation Effectiveness Frost Protected Shallow Foundations.pdf"
    elif option == "cold regions":
        file = "cold regionsREADABLE.pdf"
    elif option == "foundations issues":
        file = "foundations issuesREADABLE.pdf"
    elif option == "dewatering":
        file = "dewateringREADABLE.pdf"

```

Figure 1: Read file option to select pdf for the chatbot

For each selected PDF, the function iterates through all pages, extracting and concatenating the text content to form a comprehensive context string. This context represents the entire document relevant to the user's query.

```

51
52     # creating a pdf reader object
53     reader = PdfReader("AsifAhmed/" + file)
54
55     context = ""
56     for p in range(len(reader.pages)):
57         #creating a page object
58         page = reader.pages[p]
59
60         # extracting text from page
61         context += "\n" + page.extract_text()
62
63     return context
64

```

Figure 2: Context in the Python workspace

Contextualized Query Generation (Simplified RAG)

Unlike a typical RAG system that might employ a vector database for semantic search across a vast corpus to retrieve small, relevant chunks, this project adopts a direct contextualization method. The entire content of the user-selected document is passed to the LLM as part of the prompt. This design choice ensures that the LLM has access to the full scope of information within the specified material, minimizing the chance of missing relevant details.

Large Language Model (LLM) Integration and Constraint-Based Generation

The core intelligence of the chatbot is powered by Google's Gemini 1.5 Flash 8B model. The interaction with Gemini is carefully controlled through prompt engineering to enforce strict adherence to the provided document. At first, the `prompt_text` is meticulously crafted to guide Gemini's behavior (Figure 3)

```
72 prompt_text = f"""
73 You are a helpful assistant. Your purpose is to answer questions based ONLY on the information provided in the attached document.
74
75 If the answer to the question cannot be found directly within the document, you MUST respond with the phrase:
76 "Answer not found in document." Do not attempt to provide an answer based on your general knowledge; stick strictly to the content
77 of the document.
78
79 Document: {context}
80 Question: {user_input}
81
82 """
83
```

Figure 3: Prompt text window for the LLM integration

This explicit instruction is crucial for maintaining the academic integrity and relevance of the chatbot's responses, directly addressing the hallucination problem commonly associated with LLMs. Next step is the API configuration. The following are the settings for this step:

- `temperature=0`: This setting minimizes the randomness in Gemini's responses, encouraging more deterministic and factual output, which is highly desirable for an educational context.
- `safety settings`: Explicit safety settings are applied to block content falling under categories such as harassment, hate speech, sexually explicit content, and dangerous content, ensuring a safe and appropriate learning environment.
- `response_mime_type="text/plain"`: Ensures the output is pure text.

Finally, the client models generate content stream method is used to stream the LLM's response, providing a more responsive user experience by displaying chunks of the answer as they are generated.

Frontend Implementation with Streamlit

The Streamlit interface provides a seamless user experience for the chatbot. For example, chat history management is used to persist and display the ongoing conversation between the user and the chatbot, enhancing conversational continuity. In addition, a dropdown (`st.selectbox`) allows users to easily switch between different course documents, dynamically changing the context provided to the LLM. Moreover, user input (`st.chat_input`) captures user queries, which are then passed to the `generate` function. In addition, a "Delete History" button is provided to reset the chat session, offering users control over their data.

Implementation Details

The project is entirely implemented in Python, leveraging a standard development environment. The following are the key features for implementation:

- **Dependencies:** Key libraries include `streamlit` for the user interface (UI), `google-generativeai` for LLM interaction, `python-dotenv` for environment variable management, and `pypdf` for PDF content extraction.
- **API Key Management:** The Google Gemini API key is securely loaded from a `.env` file using `load_dotenv()` and `os.getenv()`, preventing hardcoding of sensitive credentials.
- **Modularity:** The code is structured with distinct functions (`read_file`, `generate`, `delete`) to promote readability and maintainability.

CHALLENGES

Before presenting the results, we outline the key challenges encountered during the project. As a significant portion of the effort was dedicated to developing the chatbot methodology, it is important to document the unsuccessful approaches and procedures. These insights may serve as valuable lessons for future projects of a similar nature.

Local LLM Integration Hurdles (Ollama)

An initial consideration for the project was to leverage local Large Language Models (LLMs) via platforms like Ollama. The motivation behind this approach was multi-fold: to reduce dependency on external API costs, potentially enhance data privacy by keeping processing local, and provide a more self-contained solution. However, integrating Ollama presented significant practical difficulties. Local LLMs, especially those powerful enough for effective question answering, demand substantial computational resources (CPU, RAM, and often a dedicated GPU) from the host machine. This proved to be a bottleneck for broader accessibility and deployment scenarios where users might not possess high-end hardware. Furthermore, setting up and maintaining a stable local LLM environment, managing model downloads, and ensuring seamless communication with the Streamlit frontend introduced an additional layer of complexity that was not conducive to the project's rapid development timeline. These factors led to the decision to utilize the cloud-based Google Gemini API, which abstracts away infrastructure management and resource allocation, allowing the developer to focus on prompt engineering and core chatbot functionality.

Misalignment of Tool Purpose (GitHub Copilot)

While GitHub Copilot is a powerful AI-powered code completion tool designed to assist developers in writing code faster and more efficiently, its utility for the core AI logic of this specific project was limited. Copilot excels at generating boilerplate code, suggesting syntax, and completing functions based on context. However, the unique requirement of "Asif Ahmed's AI-Answerer" was not about code generation for the AI itself, but rather about the precise configuration and interaction with an external Large Language Model (Gemini) to enforce strict domain-specific knowledge constraints. The intellectual effort was concentrated on crafting effective prompts, managing the input context, and ensuring the LLM only responded with information from provided documents. Copilot, by its design, does not contribute to the prompt engineering or the RAG paradigm's core logic, which involves instructing a separate LLM how to behave and what data to use. Therefore, while beneficial for general programming tasks, it did not directly address the fundamental AI implementation challenges of grounding the model in specific, restricted content.

Time Constraints and Advanced RAG Implementation

A significant functional limitation of the current system is the reliance on a dropdown menu for document selection and passing the entire selected document as context. While effective for a proof-of-concept, this approach is a simplified form of Retrieval-Augmented Generation (RAG). A more sophisticated RAG system would typically involve:

- **Text Chunking:** Breaking down all course materials into smaller, semantically meaningful chunks.
- **Embedding:** Converting these text chunks into numerical vector embeddings using an embedding model (e.g., a dedicated text embedding model from Google or a widely available open-source model).

- **Vector Database:** Storing these embeddings in a vector database (e.g., ChromaDB, Pinecone, FAISS) for efficient similarity search.
- **Semantic Search:** When a user asks a question, their query would also be embedded, and a semantic search performed against the vector database to retrieve only the most relevant chunks from all available course materials, regardless of the original document.
- **Contextualization:** These retrieved relevant chunks would then be fed to the LLM as context for generating the answer.

Implementing such a robust RAG pipeline would eliminate the need for the user to manually select a document from a dropdown. The chatbot would intelligently retrieve relevant information across the entire corpus of Professor Ahmed's materials. Unfortunately, integrating an embedding model, establishing a vector database, and refining the chunking and retrieval logic presented considerable undertaking requiring additional time for research, implementation, and testing. Due to the project's tight development timeline, prioritizing a functional prototype led to the adoption of the current, simpler context-passing mechanism, leaving the more advanced RAG implementation for future work.

RESULTS

The developed chatbot 'AnswerNest' is hosted at this location (<https://asif-ahmeds-ai-answerer.streamlit.app/>) which can be easily accessed by the users. This chatbot successfully demonstrates the feasibility and utility of a domain-restricted AI assistant in an educational setting.

Some of the key features of chatbot are as follows:

Accuracy and Relevance: By strictly adhering to selected materials and the explicit prompt engineering, the chatbot significantly reduces the risk of factual inaccuracies and hallucinations often seen in open-domain LLMs. This ensures that students receive information directly aligned with their course curriculum. The temperature=0 setting further enhances this determinism.

Efficiency in Information Retrieval: Students can quickly query specific topics within a chosen document from drop down menu (Figure 4), eliminating the need for manual searching through lengthy texts or video transcripts. This directly supports more efficient self-study.

Accessibility: The web-based nature of the chatbot, powered by Streamlit, makes it accessible to students from various devices without requiring complex software installations.

Controlled Knowledge Domain: The design inherently limits the chatbot's scope to the provided materials, making it a highly specialized tool rather than a general knowledge base, which is ideal for focused academic support.

Options

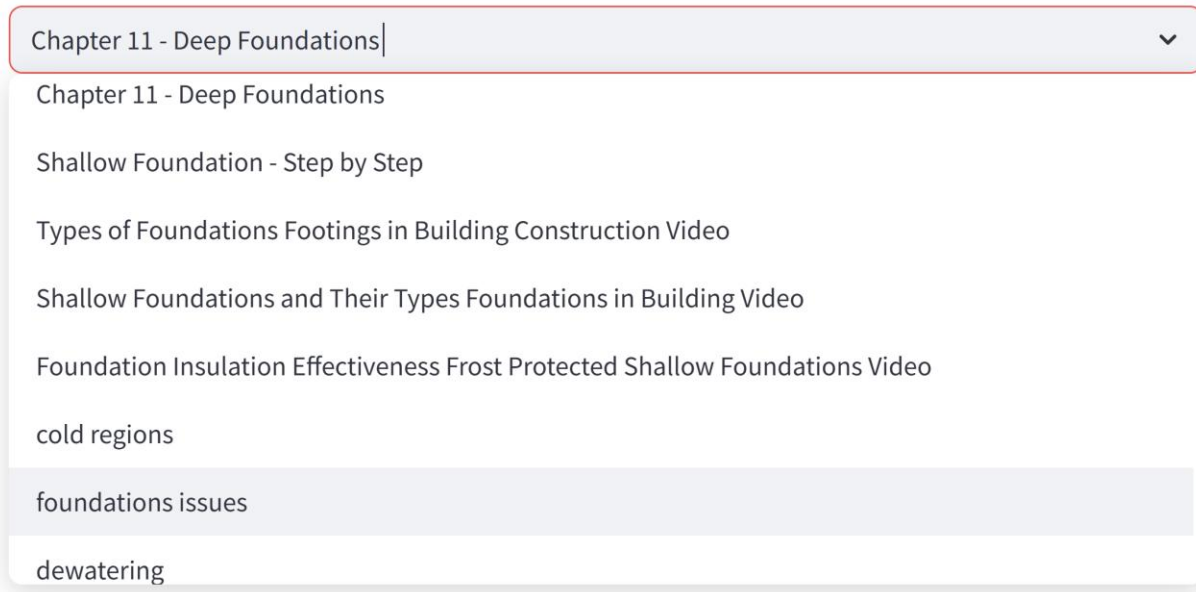


Figure 4: Drop down menu in the chatbot

4 VALIDATION OF THE CHATBOT

To evaluate the effectiveness and reliability of the developed chatbot in addressing construction-related inquiries specific to foundation engineering, a series of representative test questions were posed to the system. These questions were selected to span different topics within the domain, including deep foundations, dewatering techniques, field pumping, and shallow foundations. The chatbot's responses were captured and are presented in Figures 5 through 8, which serve as a preliminary validation of its functionality. In order to check the accuracy of the answers, the two most famous geotechnical engineering textbooks by B.M. Das and Holtz & Kovacs were used as a checking tool [23, 24].

Each response was assessed based on three primary criteria: technical correctness, contextual relevance, and clarity of explanation. For instance, in Figure 5, the chatbot provided a detailed explanation of critical items for auger cast pile construction. The answer discussed the correct grout pressure, equipment breakdown, soil stress etc. This shows that the system is capable of recognizing domain-specific terminology and delivering accurate descriptions aligned with engineering practice and literature [23].

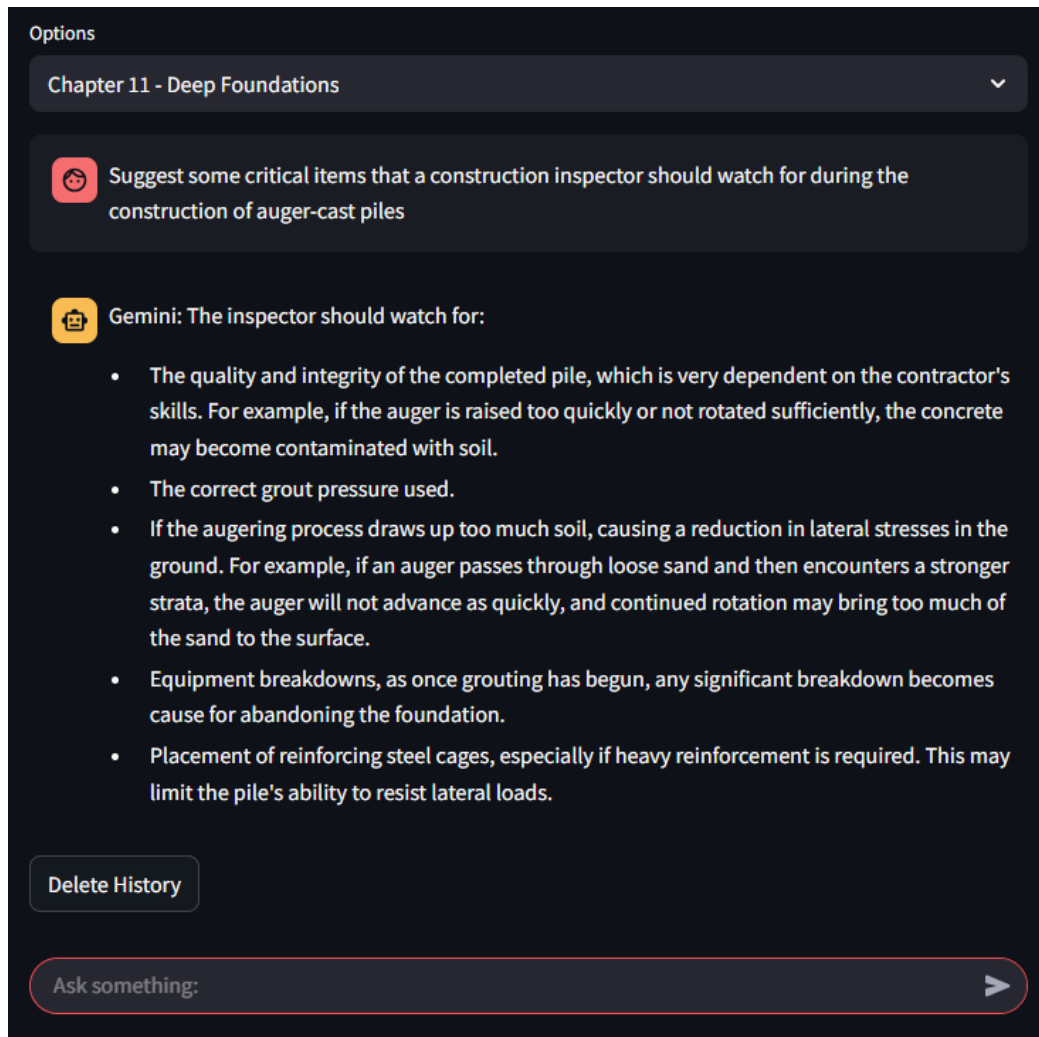


Figure 5: Deep foundation question in the chatbot

In Figure 6, the chatbot addressed a question regarding dewatering during construction. Its response identified the importance of soil type and permeability in system selection. This contextual sensitivity is critical in engineering scenarios where site conditions dictate design choices. The chatbot's ability to synthesize such considerations supports its utility as a supplemental tool in decision-making and learning environments.

The field pumping question shown in Figure 7 further demonstrates the chatbot's capacity to interpret field procedures and offers insights. Its response included a general method for conducting a field pumping test, which is consistent with standard procedures outlined in geotechnical engineering manuals and academic sources [24].

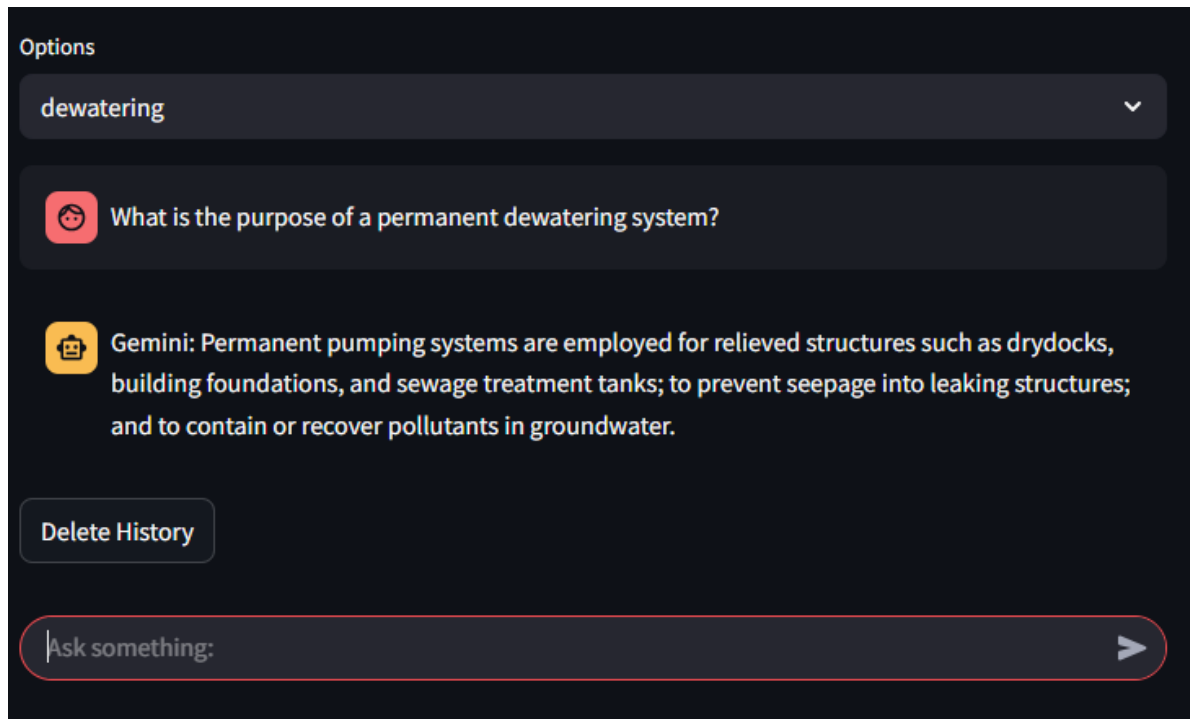


Figure 6: Dewatering during construction question in the chatbot

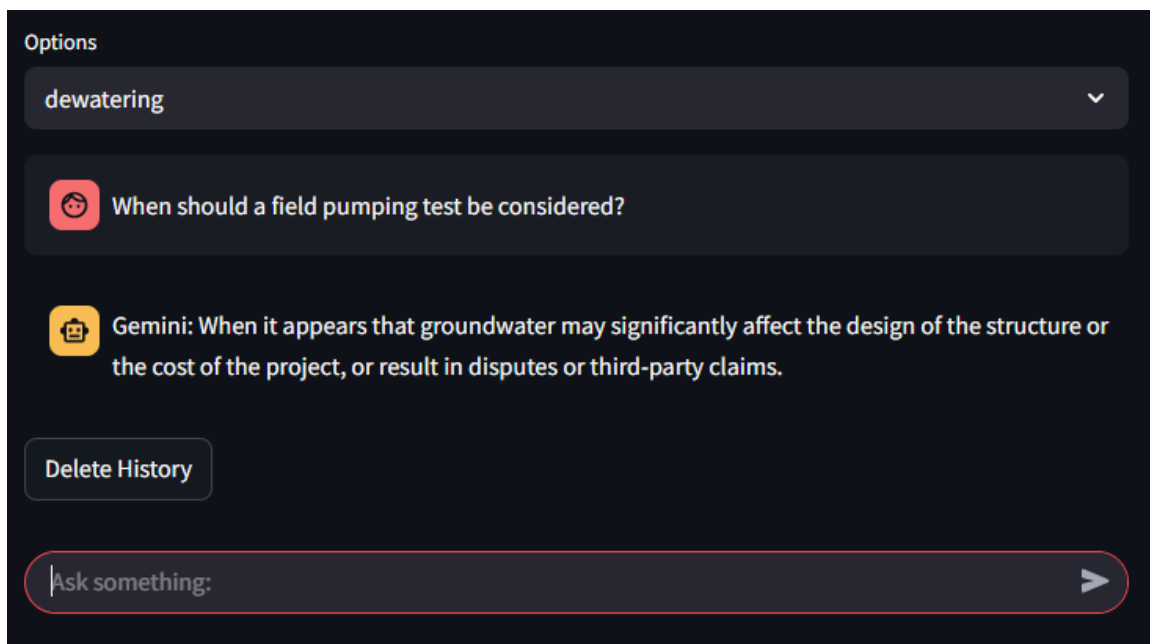


Figure 7: Field pumping related question in the chatbot

Lastly, the chatbot's reply to the shallow foundation query in Figure 8 illustrated sound knowledge of various shallow footing. It should be mentioned that- this answer was selected from a video transcript. As mentioned in the methodology section, in addition to technical pdf, the chatbot was also provided a few youtube video transcripts. This example highlights the model's ability to apply foundational theory to practical questions.

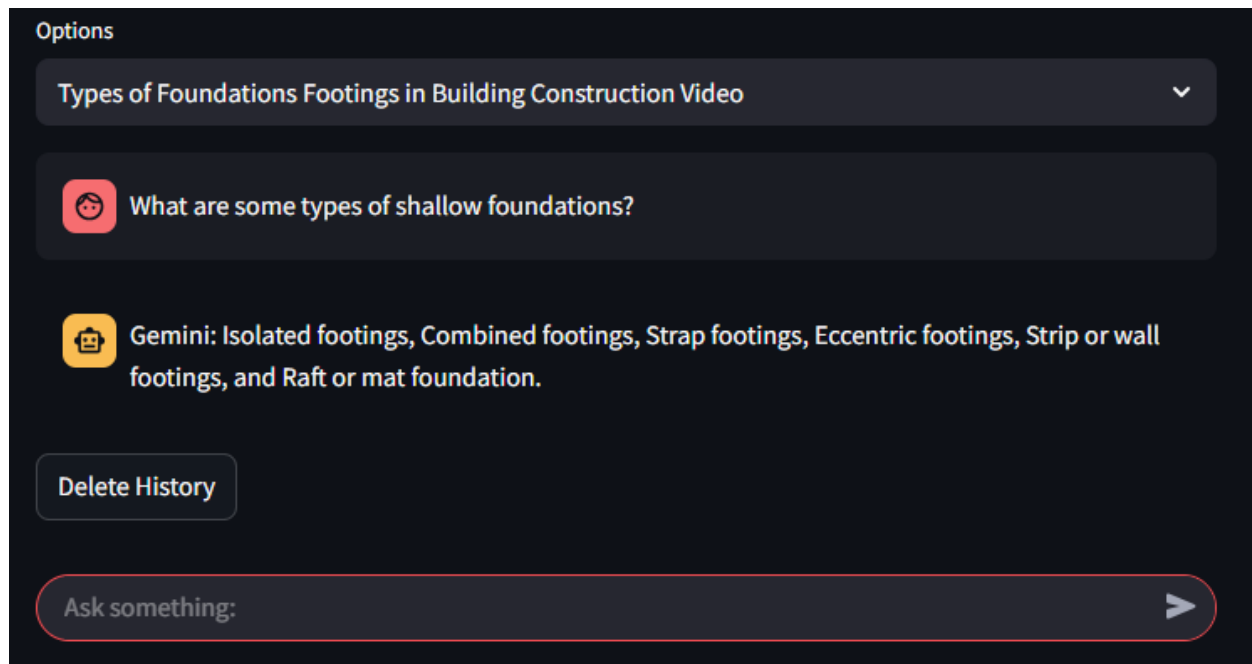


Figure 8: Shallow foundation question in the chatbot

Collectively, the chatbot's performance across these examples suggests that it can accurately interpret and answer construction-related questions in foundation engineering. The responses exhibit both surface-level correctness and deeper understanding, indicating that the underlying language model has effectively internalized relevant technical knowledge.

LIMITATIONS OF THE STUDY

Despite its strengths, the current implementation has certain limitations. First of all, the chatbot can only process and respond based on the entirety of a single selected document at a time. It cannot currently synthesize information across multiple documents simultaneously or from a fragmented knowledge base. This is a simplification of a full RAG system which would typically involve retrieving relevant chunks from a larger indexed corpus. Another issue is- while beneficial for preventing hallucinations, the strict "Answer not found in document" response can sometimes be too abrupt if the answer requires slight inference or is spread across disparate sections of a large document. In addition, for extremely large documents, passing the entire content as context might approach or exceed the LLM's token limit, requiring more advanced chunking and retrieval strategies. Moreover, adding new materials requires manual conversion to PDF and updating the read_file logic.

CONCLUSIONS

This research successfully developed and validated "Asif Ahmed's AI-Answerer," a web-based educational chatbot designed to enhance the learning experience in geotechnical engineering courses. By leveraging Streamlit for an accessible user interface and Google's Gemini LLM for conversational AI, the chatbot effectively provides accurate and contextually relevant answers strictly derived from Professor Ahmed's curated course materials. The implementation of a simplified Retrieval-Augmented Generation (RAG)

approach, alongside meticulous prompt engineering, proved crucial in ensuring the chatbot's fidelity to the provided documents and mitigating issues of AI hallucination, thereby maintaining academic integrity.

The preliminary validation demonstrated the chatbot's capability to answer complex, domain-specific questions on topics such as deep foundations and dewatering, directly supporting students in understanding the practical implications of their designs. While challenges such as integrating local LLMs like Ollama and the limited utility of general code assistants like GitHub Copilot were encountered, the project successfully delivered a functional prototype. Furthermore, the decision to defer a more advanced embedding-model-based RAG system due to time constraints highlighted an area for significant future enhancement. This project serves as a valuable proof-of-concept, demonstrating how specialized AI tools, grounded in instructor-provided content, can effectively bridge the gap between theoretical knowledge and practical construction challenges in civil engineering education, preparing students more comprehensively for their professional careers.

ACKNOWLEDGMENTS

The authors acknowledge the use of OpenAI's GPT-4 language model to assist in generating narrative structure, improving clarity in technical writing, and rephrasing sections of the manuscript. The authors maintained full responsibility for the study design, data interpretation, and final content, and used the model strictly for editorial and linguistic support. All technical analysis, modeling, and results were produced and verified independently by the authors.

AUTHOR CONTRIBUTIONS

The authors confirm contribution to the paper as follows: study conception and design: Asif Ahmed, Andrew Fagan; data collection: Andrew Fagan; analysis and interpretation of results: Andrew Fagan; draft manuscript preparation: Asif Ahmed, Andrew Fagan. All authors reviewed the results and approved the final version of the manuscript.

DECLARATION OF CONFLICTING INTERESTS

The authors declared no potential conflicts of interest with respect to the research, authorship, and/or publication of this article.

FUNDING

The project is funded by Summer Undergraduate Research program (SURP) at SUNY Poly.

REFERENCES

1. Bae, H., Polmear, M., & Simmons, D. R. (2022). Bridging the gap between industry expectations and academic preparation: Civil engineering students' employability. *Journal of Civil Engineering Education*, 148(3), 04022003.

2. Wu, W., Tesei, A., Ayer, S., London, J., Luo, Y., & Gunji, V. (2018, October). Closing the skills gap: Construction and engineering education using mixed reality—A case study. In *2018 IEEE Frontiers in Education Conference (FIE)* (pp. 1-5). IEEE.
3. Suprun, E., Perisic, N., Stewart, R., & Mostafa, S. (2019). Preparing the next generation of civil engineering graduates: identifying and combating the digital skills gap. In *30th Annual Conference for the Australasian Association for Engineering Education (AAEE 2019)*. Australasian Association for Engineering Education (AAEE).
4. Infrastructure Investment and Jobs Act, H.R. 3684, 117th Cong. (2021).
<https://www.congress.gov/bill/117th-congress/house-bill/3684>
5. Bureau of Labor Statistics. (2022). “Occupational Outlook Handbook: Construction Laborers and Helpers,” Available at: <https://www.bls.gov/>
6. New York State Office of Strategic Workforce Development (2025). Available at
<https://esd.ny.gov/office-strategic-workforce-development>
7. Wollny, S., Schneider, J., Di Mitri, D., Weidlich, J., Rittberger, M., & Drachsler, H. (2021). Are we there yet?-a systematic literature review on chatbots in education. *Frontiers in artificial intelligence*, 4, 654924.
8. Neumann, A. T., Conrardy, A. D., & Klamma, R. (2021, November). Supplemental mobile learner support through moodle-independent assessment bots. In *International Conference on Web-Based Learning* (pp. 75-89). Cham: Springer International Publishing.
9. Ruan, S., Jiang, L., Xu, J., Tham, B. J. K., Qiu, Z., Zhu, Y., ... & Landay, J. A. (2019, May). Quizbot: A dialogue-based adaptive learning system for factual knowledge. In *Proceedings of the 2019 CHI conference on human factors in computing systems* (pp. 1-13).
10. Meyer von Wolff, R., Nörtemann, J., Hobert, S., & Schumann, M. (2019, November). Chatbots for the information acquisition at universities—a student’s view on the application area. In *International Workshop on Chatbot Research and Design* (pp. 231-244). Cham: Springer International Publishing.
11. Yang, S., & Stansfield, K. (2022, January). AI chatbot for educational service improvement in the post-pandemic era: A case study prototype for supporting digital reading list. In *Proceedings of the 2022 13th International Conference on E-Education, E-Business, E-Management, and E-Learning* (pp. 24-29).
12. Hobert, S. (2019). Say hello to ‘coding tutor’! design and evaluation of a chatbot-based learning system supporting students to learn to program.
13. Okonkwo, C. W., & Ade-Ibijola, A. (2020). Python-bot: A chatbot for teaching python programming. *Engineering Letters*, 29(1).
14. Pham, X. L., Pham, T., Nguyen, Q. M., Nguyen, T. H., & Cao, T. T. H. (2018, November). Chatbot as an intelligent personal assistant for mobile language learning. In *Proceedings of the 2018 2nd International Conference on Education and E-Learning* (pp. 16-21).
15. Pursnani, V., Sermet, Y., Kurt, M., & Demir, I. (2023). Performance of ChatGPT on the US fundamentals of engineering exam: Comprehensive assessment of proficiency and potential implications for professional environmental engineering practice. *Computers and Education: Artificial Intelligence*, 5, 100183.
16. VarastehNezhad, A., Tavasoli, R., Masumi, M., & Taghiyareh, F. (2024, October). Llm performance assessment in computer science graduate entrance exams. In *2024 11th International Symposium on Telecommunications (IST)* (pp. 232-237). IEEE.

17. Maitland, A., Fowkes, R., & Maitland, S. (2024). Can ChatGPT pass the MRCP (UK) written examinations? Analysis of performance and errors using a clinical decision-reasoning framework. *BMJ open*, 14(3), e080558.
18. Honig, C., Rios, S., & Desu, A. (2025). Generative AI in engineering education: understanding acceptance and use of new GPT teaching tools within a UTAUT framework. *Australasian Journal of Engineering Education*, 1-13.
19. Xu, H. R., Zhang, N., Yin, Z. Y., & Njock, P. G. A. (2025). GeoLLM: A specialized large language model framework for intelligent geotechnical design. *Computers and Geotechnics*, 177, 106849.
20. Wu S, Otake Y, Mizutani D, Liu C, Asano K, Sato N, Saito T, Baba H, Fukunaga Y, Higo Y, Kamura A, Kodama S, Metoki M, Nakamura T, Nakazato Y, Shioi A, Takenobu M, Tsukioka K, Yoshikawa R. Future-proofing geotechnics workflows: accelerating problem-solving with large language models. *Georisk*. 2024. <https://doi.org/10.1080/17499518.2024.2381026>.
21. Streamlit: A Faster Way to Build and Share Apps - Simplilearn.com. (2025-06-19). [Accessed July 29, 2025].
22. Generative AI | Google Cloud. [Accessed July 29, 2025].
23. Das, B. M., and Sivakugan, N. (2018). *Principles of foundation engineering*. Cengage learning.
24. Holtz, R. D., and Kovacs, W. D. (1981). *An introduction to geotechnical engineering*. Prentice-Hall.